

object inside the function would also exist outside the function, making this modification a side effect.

A related concept, a *deterministic function*, always returns the same return value given the same arguments. The `subtract(123, 987)` function call always returns `-864`. Python's built-in `round()` function always returns `3` when passed `3.14` as an argument. A *nondeterministic function* won't always return the same values when passed the same arguments. For example, calling `random.randint(1, 10)` returns a random integer between `1` and `10`. The `time.time()` function has no arguments, but it returns a different value depending on what your computer's clock is set to when the function was called. In the case of `time.time()`, the clock is an external resource that is effectively an input into the function the same way an argument is. Functions that depend on resources external to the function (including global variables, files on the hard drive, databases, and internet connections) are not considered deterministic.

One benefit of deterministic functions is that you can cache their values. There's no need for `subtract()` to calculate the difference of `123` and `987` more than once if it can remember the return value from the first time it's called with those arguments. Therefore, deterministic functions allow us to make a *space-time trade-off*, quickening the runtime of a function by using space in memory to cache previous results.

A function that is deterministic and free of side effects is called a *pure function*. Functional programmers strive to create only pure functions in their programs. In addition to those already noted, pure functions offer several benefits:

- They're well suited for unit testing, because they don't require you to set up any external resources.
- It's easy to reproduce bugs in a pure function by calling the function with the same arguments.
- Pure functions can call other pure functions and remain pure.
- In multithreaded programs, pure functions are thread-safe and can safely run concurrently. (Multithreading is beyond the scope of this book.)